



FACULTY OF INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING

Ville-Veikka Kiiskilä

**EMPLOYING DEVICE INVENTORY AND
FAILURE DATA FOR TEST CONFIGURATION
DISCOVERY AND DEVICE UTILIZATION**

Bachelor's Thesis
Degree Programme in Computer Science and Engineering
May 2025

Kiiskilä V. (2025) Employing Device Inventory and Failure Data for Test Configuration Discovery and Device Utilization. University of Oulu, Degree Programme in Computer Science and Engineering, 27 p.

ABSTRACT

Within network integration testing, device capacity is a key constraint for comprehensive testing. This thesis investigates how fault tickets and device inventory data could be utilized to automatically discover test configurations, increase device utilization, and aid decision making using a two-prong approach. Firstly, the research focuses on examining and developing methods for interpreting and analyzing natural language data found in fault tickets, with the goal of converting unstructured natural language into structured data. The system can then automatically identify the available test configurations and their compatibility based on the underlying devices. Secondly, the device inventory interface and the supporting infrastructure offer data from which utilization, failure rate, and conflicts can be calculated. This data will also be used to create a reporting system that can show reservation conflicts, devices, and their utilization. Resulting in more efficient configuration discovery and device utilization. The fine-tuned T5-base large language model (LLM) showed over a 10% increase in accuracy, compared to regular expressions (Regex), with a 94% accuracy on the test dataset. The developed application could effectively deduce configuration compatibility by utilizing the extraction process. Furthermore, a recommendation for organizing permissions and ownership within the device inventory was created.

Keywords: fault tickets, natural language analysis, structured data

Kiiskilä V. (2025) Laitteinventaarion ja vikatietojen hyödyntäminen testikonfiguraatioiden tunnistamiseen ja laitteiden tehokkaampaan käyttöön. Oulun yliopisto, Tietotekniikan tutkinto-ohjelma, 27 s.

TIIVISTELMÄ

Laitekapasiteetti on keskeinen rajoite kattavassa verkkointegraatiotestauksessa. Tämä tutkielma käsittelee, miten vikatikettien ja laiteinventaarion tietoja voidaan hyödyntää laitteiden käyttöasteen tehostamiseen, testikonfiguraation löytämiseen ja päätöksenteon avustamiseksi. Ensimmäiseksi tutkimus keskittyy tarkastelemaan ja kehittämään menetelmiä luonnollisen kielen datan tulkintaan, jotta rakenteettomasta luonnollisesta kielestä voidaan tulkita niiden taustalla olevat testikonfiguraatiot. Tämän jälkeen järjestelmä voi automaattisesti tunnistaa niiden yhteensopivuuden suhteessa käytössäoleviin laitteisiin. Toiseksi laiteinventaarion rajapinta ja muut rajapinnat tarjoavat dataa, josta voidaan laskea käyttöaste, jonotusajat ja epäonnistumisprosentit. Tätä dataa käytetään raportointijärjestelmän luomiseen, joka päättelee resurssipoolien varauskonfliktit sekä näyttää vapaat konfiguraatiot ja niiden varaukset. Hienosäädetyin T5-base kielimallin tarkkuus parani yli 10% verrattuna Regex-pohjaiseen ratkaisuun. Lopullinen malli saavutti noin 94% tarkkuuden testiaineistolla. Kehitetty applikaatio pystyi näyttämään konfiguraation yhteensopivuuden käyttäen tätä luokittelu-menetelmää. Lisäksi luotiin suositus käyttöoikeuksien ja omistusoikeuksien järjestämiseen laiteinventaariorissa.

Avainsanat: vikatiketit, luonnollisen kielen analyysi, rakenteellinen tieto

TABLE OF CONTENTS

ABSTRACT

TIIVISTELMÄ

TABLE OF CONTENTS

FOREWORD

LIST OF ABBREVIATIONS AND SYMBOLS

1. INTRODUCTION.....	7
1.1. Author's Contributions and the Role of Artificial Intelligence	7
2. BACKGROUND AND RELATED WORK	8
2.1. Information Extraction and Named Entity Recognition.....	8
2.2. Rule-Based Approach	8
2.3. Supervised Learning	9
2.4. Extracting Structured Data with Large Language Models	10
2.5. LLM Frameworks for Structured Data Extraction	10
2.6. T5 Model	10
2.7. T5 Model in Named Entity Recognition	11
3. DATA SOURCES	12
3.1. Device Inventory System.....	12
3.2. Fault Ticket Service	12
3.3. Test Automation Framework.....	13
3.4. Configuration Service	13
3.5. Version Control System.....	13
4. DEVICE EXTRACTION	15
4.1. Training Dataset Creation.....	15
4.2. LLM Fine-Tuning	15
4.3. Context Reduction	16
4.4. Evaluation Methodology	16
4.5. Evaluated Models	17
4.6. Analysis	17
5. APPLICATION	19
5.1. Application Infrastructure.....	19
6. DEVICE INVENTORY.....	22
7. DISCUSSION	23
8. SUMMARY	24
9. REFERENCES	25

FOREWORD

I would like to thank my supervisor Jaakko Suutala for his guidance and support during the completion of this thesis. I also would like to thank the Nokia L1 Call team for the exciting thesis topic and the assistance they have provided.

Oulu, May 2nd, 2025

Ville-Veikka Kiiskilä

LIST OF ABBREVIATIONS AND SYMBOLS

API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BNER	Biological Named Entity Recognizer
C4	Colossal Clean Crawled Corpus
CI/CD	Continuous Integration and Continuous Deployment
CNN	Convolutional Neural Network
DNS	Domain Name System
GUI	Graphical User Interface
GPU	Graphics Processing Unit
IP	Internet Protocol
LLM	Large Language Model
NER	Named Entity Recognition
NLP	Natural language Processing
Regex	Regular Expression
SFT	supervised fine-tuning
T5	Text-To-Text Transfer Transformer
μ	Population Mean
σ	Population Standard Deviation

1. INTRODUCTION

The increasing number of users, higher bandwidth requirements, and the growth of mobile data traffic create challenges in wireless cellular networks [1]. Along with 4G, 5G brings new possibilities and technologies such as millimeter wave communication, spectrum sharing, and more advanced network slicing. As networks grow and increase in complexity, comprehensive testing is vital for network performance and stability. Testing capacity remains a key limitation constrained by personnel capacity and resource availability. For this reason, a device inventory has been adopted on a smaller scale, but in the future even more widespread adoption is expected. In integration testing, orchestrating real devices can be problematic as they have to be reserved automatically and efficiently. Resource-pooling devices can aid this, but deduction processes with conflicts grow in complexity. This thesis aims to reduce manual processes through an application that interfaces with multiple data sources. The application can reduce manual labor by giving testability scores for fault tickets, and it can also offer insights into device utilization and conflicts between reservations. This can free up testing capacity through more efficient device usage and data-driven decision-making, as well as personnel resources by reducing manual work. The fault tickets contain test configurations in natural language which, due to its unstructured nature, is hard to analyze. Large language models have shown promise in extracting chemical compounds and locations from natural language [2] [3]. This research therefore investigates whether LLMs can be effective in extracting test configurations from unstructured text. It specifically compares LLM-based device extraction against more traditional Regex-based extraction methods, and investigates whether fine-tuning enhances the extraction performance. In addition, the study will explore the limitations and considerations that should be taken into account with larger adoption of the device inventory.

1.1. Author's Contributions and the Role of Artificial Intelligence

In this thesis Overleaf's Writiful integration was utilized within the limitations of their free tier. It offers and forces simple writing suggestions. In addition, GitHub Copilot was used for debugging, code completion, and to a limited extent in code generation. Artificial intelligence (AI) has also been explored for the creation of synthetic training data. This thesis and the related code are still primarily written by the author.

2. BACKGROUND AND RELATED WORK

Test configuration discovery in this context is highly particular and proprietary to Nokia, hence there is a lack of related literature. This literature review will instead focus on the problem of natural language processing (NLP) and extracting natural language to structured data, which is more widely and publicly researched.

2.1. Information Extraction and Named Entity Recognition

Information extraction is a sub-task of text mining and one of the key problems within it [4]. The goal of information extraction is to discover structured information from unstructured text like natural language. After extraction, structured data can be more easily used in databases, search engines, and for further applications. Within information extraction named entity recognition (NER) is a way to identify named entities from unstructured text and match them into predefined classes or types. NER is a fundamental part of information extraction and a key building block for further applications. NER will be a key focus in this thesis, since the test configuration devices need to be extracted, counted, and classified.

2.2. Rule-Based Approach

In most cases, some part of NER can be achieved with a rule-based approach, in which the text is compared against a set of rules and the corresponding rule is activated when a match is found [4]. The rules consist of a pattern and a resulting action. These kinds of rule-based approaches are widely used and there are many different tools that facilitate creating and using these rules. For example, regular expressions offer a way to solve complex string-matching problems [5]. Although, regular expressions are widely used even to the point of being built into many programming languages. They are not unproblematic; Regex patterns are notoriously difficult to understand. Especially when the patterns increase in complexity. For example, the following Pythonic regular expression finds the quantity and device from a string like "2 pcs of deviceA100".

```
rf"(\d+)\s*(?:pcs|pieces)\s*of\s*({expected_device})"
```

Here (d+) matches one or multiple digits, s* handles white-spaces, and (?:pcs | pieces) creates a non-capturing group for pieces or pcs. Still there are a few faults with this statement. Firstly, it is case-sensitive, meaning that a very common misspelling: "2 pcs Of deviceA100" would not be recognized. Secondly, it would not recognize the model "A100" if it was written slightly differently with spaces: "2 pcs of device A100". Both of these faults can be fixed with additional pattern components:

```
rf"(?i)(\d+)\s*(?:pcs|pieces)\s*of\s*{
    expected_device.replace("'", r'\s*')}
"
```

Most of the faulty cases could be fixed with additional Regex patterns to fit most cases even though the dataset is large and quite diverse. Table 1 presents some other variations that could be fixed with additional Regex statements. The problem Regex cannot solve with more Regex is context clues and reasoning. Some of the data requires simple logical reasoning, for instance "A bag can fit five items, the bag is full of apples. How many apples are there?", represents a problem that is hard to solve with Regex and instead requires simple logical reasoning. Another problem is the missing context; for example, many fault tickets list the devices used and then refer to these same devices when describing how they are connected to each other: "4xDeviceA and 2xDeviceB (Two deviceA's are connected to a corresponding DeviceB)". Regex patterns would count the connection description into the device counts. There are also problems where Regex mistakes version numbers for device counts.

Table 1. Table of example inputs, registered quantities and devices

Input	Quantity	Expected device
2 pcs of deviceA100	2	deviceA100
3 pcs Of device A100	3	deviceA100
Three pcs Of device A102	None	deviceA102
3xdeviceB102+4*deviceA	None	deviceB102
A pair of device B102's	None	deviceB102

2.3. Supervised Learning

In comparison to the rule-based approaches which require explicitly defined patterns, supervised learning can provide an alternative way of achieving NER [4]. Supervised learning instead requires a dataset with inputs and the corresponding outputs. An algorithm is then trained on these data to predict the outputs for unseen data points. One supervised learning technique is fine-tuning. In supervised fine-tuning (SFT) a pre-trained model is given domain-specific data to further specialize the LLM for a specific task. In essence, fine-tuning transfers knowledge from the dataset into the pre-trained model, improving its performance on that specific task like text classification, question answering, or data extraction [6]. In comparison to the rule-based approaches which require explicitly defined patterns, supervised learning can provide an alternative way of achieving NER [4]. Supervised learning instead requires a dataset with inputs and the corresponding outputs. An algorithm is then trained on these data to predict the outputs for unseen data points. One supervised learning technique is fine-tuning. In supervised fine-tuning (SFT) a pre-trained model is given domain-specific data to further specialize the LLM for a specific task. In essence, fine-tuning transfers knowledge from the dataset into the pre-trained model, improving its performance on that specific task like text classification, question answering, or data extraction [6].

2.4. Extracting Structured Data with Large Language Models

Large language models have shown promise in information extraction [7]. In one research paper, a large language model was utilized to make biological named entity recognizer (BNER). BNER could, with high degree of precision, identify proteins, genes, and molecules from scientific texts. These details were extracted to structured formats with an F1 score of 88.7% where the F1 score is the harmonic mean of precision and recall. Similarly, Zhang et al. researched how LLMs could extract chemical compounds from natural text [2]. Fine-tuned models performed better than their prompt-only counterparts and even approached 90% F1 scores with proper training data. Achieving a true structured format with an LLM is not unproblematic [8]. In the study, different LLMs are evaluated on how they obey when asked to answer in a purely structured output, which means no redundant information only the specified information in the correct format. The performance of different LLMs varied. One key finding was that the model size did not necessarily lead to better performance in this task; Llama 13b did not outperform the smaller Llama 7b model, although the same was not true for Qwen model series. For this reason, this case study will evaluate multiple model sizes. Through prompt-engineering, fine-tuning, and frameworks, large language models have also been adjusted to output the structured format needed for further data manipulation. The first aforementioned study used prompt engineering [7]. In this problem, fine-tuning was beneficial as it combined the domain specific information and the structured output format.

2.5. LLM Frameworks for Structured Data Extraction

There exists several wrappers and frameworks for extracting structured data from large language models. Some popular are Kor [9], Jsonformer [10], and LangChain [11]. These work on various high-level concepts to generate output in a structured format. In prompt engineering, the prompt in some way includes the suggestion that the output should be in some structured format. In zero-shot prompting the LLM is not provided with any examples on how to output in this structured format. In the opposite case, if the LLM is provided with examples it is few-shot prompting [8]. Kor is designed around this prompt-based parsing [9]. Langchain also supports this prompt-based parsing, but in addition fine-tuned function calling similar to prefixing and JSON mode where the model is forced to output in structured format through postprocessing. JsonFormer is a different type of approach. It utilizes the fixed structure of the output, and only generates the content tokens [10][11]. From these, fine-tuning was chosen since it tackles two problems at once, the extraction of devices correctly and the output structured output format [6][8]. During fine-tuning, the model is able to learn the device names and the structure of configurations.

2.6. T5 Model

T5 is a series of large language models developed by Google, which includes different model sizes and model versions, such as T5-base, T5-Flan, and many other variants

[12]. T5 models are **text-to-text** models based on **transformers** and **transfer learning** hence the name T5. Since T5 is a text-to-text model. The inputs and outputs are given in text form, and the model then predicts the desired text as an output. T5 is trained on the "Colossal Clean Crawled Corpus" (C4) dataset which is created from the Common Crawl dataset via a set of heuristic rules. In the transfer learning phase, words from the dataset are masked, and the model is then prompted to guess the missing words. In comparison to earlier models like BERT, **B**idirectional **E**ncoder **R**epresentations from **T**ransformers [13], T5 can predict multiple tokens at a time. This sequence prediction is important since many words might have different meanings when grouped together. For instance, "river bank" and "bank robbery" have very different meanings. This combined with one of the core concept of transformers called self-attention, that weighs word sequences, allows the T5 to model complicated language structures. T5 is then trained on task-specific prefixes, such as translating, summarizing, or evaluating sentence acceptability. These help the model pick the appropriate subset of weights. In Figure 1, a light visualization of model weights is shown after prefixing for task-specific functionalities. Prefixes will be utilized during the fine-tuning process for device extraction. T5-Flan is an improved version of the basic T5 model, fine-tuned for a mixture of tasks such as chain-of-thought fine-tuning, instruction fine-tuning, and multitask instruction fine-tuning [12] [14].

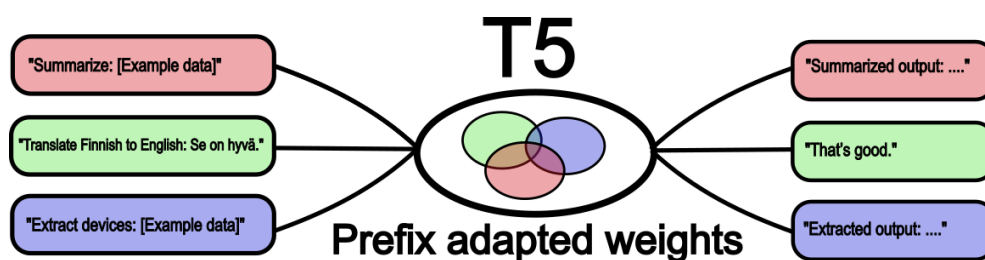


Figure 1. Visualization on how prefixes affect model weights.

2.7. T5 Model in Named Entity Recognition

Although T5 is not specifically meant for NER, it has shown promise in extracting chemical compounds and location names from social media posts [2] [3]. In the location extraction, transformer-based T5 was compared against SpaCy's approach involving convolutional neural networks (CNNs). CNNs have a simpler architecture that allows for faster training and computations but introduce drawbacks in processing diverse location representations. Where transformers process longer sequences of text, CNNs are focused on individual words through a token-based approach. The fine-tuned T5 model achieved a 93% accuracy in testing, which significantly outperformed the CNN-based method with 41% accuracy. The study also highlighted the T5 models ability to understand and extract location names accurately, even from complex variations. These kinds of complex variations correlate well with the device extraction dataset which features similar characteristics. From the transformer-based models T5 was specifically chosen for its size, Github documentation, supporting libraries, and permissive license [15] [16].

3. DATA SOURCES

By combining multiple different data sources, the application can aid decision-making and improve workflows. Application programming interfaces (API) offer routes through which data can be fetched and manipulated. Programming interfaces are the key to combining these different data sources into a single application, and they also help with other task, such as creating the training data. Figure 2 shows a preliminary illustration of the connections between the different data sources and Table 2 summarizes the data that the sources provide. Next, different services providing the data sources are introduced.

3.1. Device Inventory System

Device inventory is a digital system with a database, a graphical user interface (GUI), and an API. Currently, only a fraction of the devices are listed inside the device inventory, but in the future there is going to be greater adoption of this technology.

The inventory is based on topologies and reservations. In essence, topologies are blueprints with devices and their respective details like connections, internet protocol (IP) address, and other related details. These topologies are then reserved for either manual or automatic testing. When reserved, the topology is locked and other conflicting reservations are put into a queue. All the devices inside the topology need to be available for the reservation to succeed unless there is an abstracted device. Abstract devices are basically a resource pool from which the next available device can be utilized. In contrast, one normal device can only be in one topology simultaneously.

The device inventory offers many key benefits, such as the aforementioned abstract devices, usage statistics, and easier automation. This thesis especially utilizes the API through which devices are extracted and usage is tracked. One of the goals is also to find improvement ideas related to device and topology entry; what details are needed to further enable the usage of the device inventory. From the device inventory, structured information about topologies, reservations, active conflicts, and device utilization are utilized for the application.

3.2. Fault Ticket Service

Fault tickets are recorded into a unified system. This service provides API access to fault tickets, which are either internal or customer-generated forms that detail which kind of configuration had what kind of fault. These are often quite detailed with device logs, software versions, hardware details, and configuration details. These fault tickets are natural language unlike the device inventory data which is structured data. The device information can be extracted through an API via a related field, but the crux of the problem is understanding this natural language and extracting it into structured data, which can then be further used for analysis. For the application, the date, id and test configuration description are utilized.

3.3. Test Automation Framework

Test automation framework is a system that facilitates our testing. The pipeline offers an API to its data, such as testing logs, reservation details, and testing jobs. Scheduled quality testing jobs can be linked to the underlying device data via the device inventory reservation. This works as a uniquely identifiable signal. Connecting these data sources also gives us deeper insight into the fault tickets since the test made inside the continuous integration and continuous deployment (CI/CD) pipeline contributes towards the fault tickets. When a problem is found, it is reported via a fault ticket.

3.4. Configuration Service

This service specializes in documenting test configurations in greater detail. Although most fault tickets contain only natural language, some also include a configuration identifier. This identifier can be used to retrieve the configuration programmatically from the configuration database. This data source could offer a way to create the training data almost automatically, by first gathering the fault tickets with both natural language configuration data and configuration id. A simple script can then be utilized to create the training data example outputs by fetching the underlying configuration by its id. This data will have to be manually verified, but it could greatly reduce the effort needed to create the training dataset. The configuration data includes details about the devices within the configuration, as well as their connections, usage, and any modifications related to the test configuration. This is important since the configuration can change between the creation of the fault ticket and the creation of the training data.

3.5. Version Control System

Version control is a widely used system in modern software development. It offers robust change tracking and traceability, enhanced collaboration, and backup solutions [17]. The testing framework utilizes version control for continuous integration and continuous deployment. As a data source, version control includes the testing configurations, and they are key to utilization and configuration availability analysis, since they can be sorted by type through the version control API.

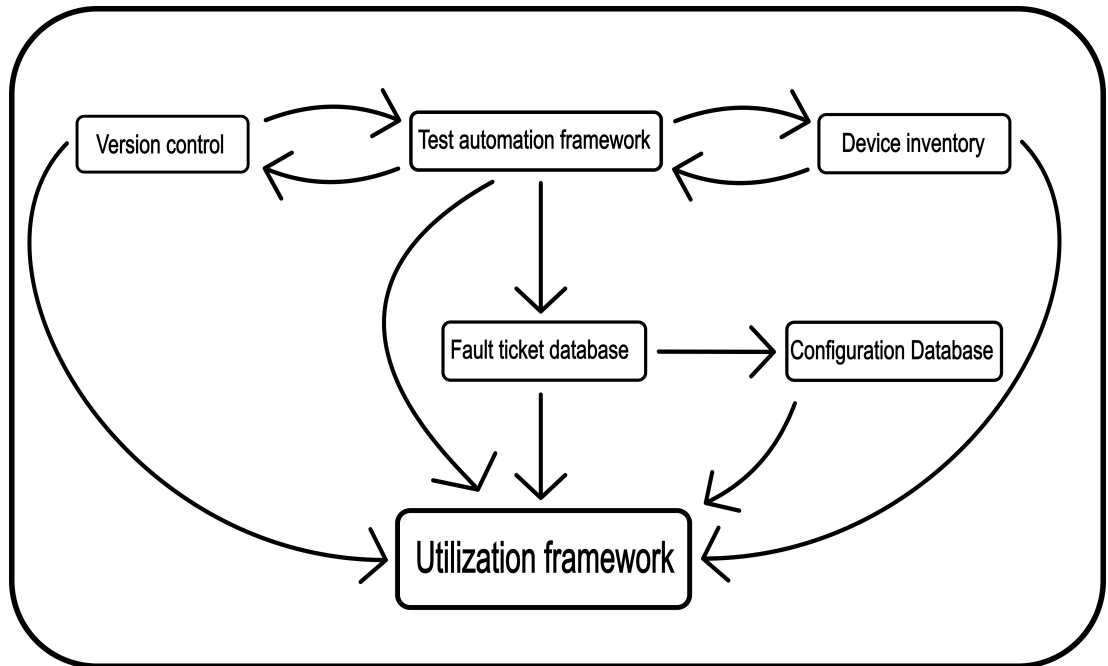


Figure 2. Illustrative visualization on how information flows through the data sources.

Table 2. Summary of the data sources and the information they provide

Data source	Provided data
Device Inventory System	Device utilization, topologies, active conflicts, and reservations
Fault Ticket Service	Ticket names, dates, and device configurations
Test Automation Framework	Test configuration failure data and test durations
Configuration Service	Test configurations, and their usage and modifications
Version Control System	Test configurations and their categories

4. DEVICE EXTRACTION

This chapter investigates the performance of different models and methods for the device extraction, with the device extraction being a key research question. From Chapter 2 the rule-based approach was implemented with Pythonic Regex and SFT through fine-tuning variations of the T5 model.

4.1. Training Dataset Creation

The initial plan for training data creation was to link fault tickets with configuration database by a configuration identifier found in some fault tickets. This is depicted in Figure 3. This proved to be difficult, as only a small fraction of configurations found in the database exactly matched the fault ticket. Most fault tickets list the minimum viable testing equipment while the configuration database lists the whole test line. As a result of this, the dataset had to be created manually, which severely limited the size. The dataset was created using randomly selected fault tickets with a preprocessing step where only the configuration info was kept. Then the initial devices were extracted and the correct amounts were manually changed. Curating or balancing the dataset has shown to have positive effects on results by improving generalization and enhancing model accuracy [18][19]. Since the extraction part was not the only focus of this thesis, further optimizations like dataset curation and hyperparameters were left out.

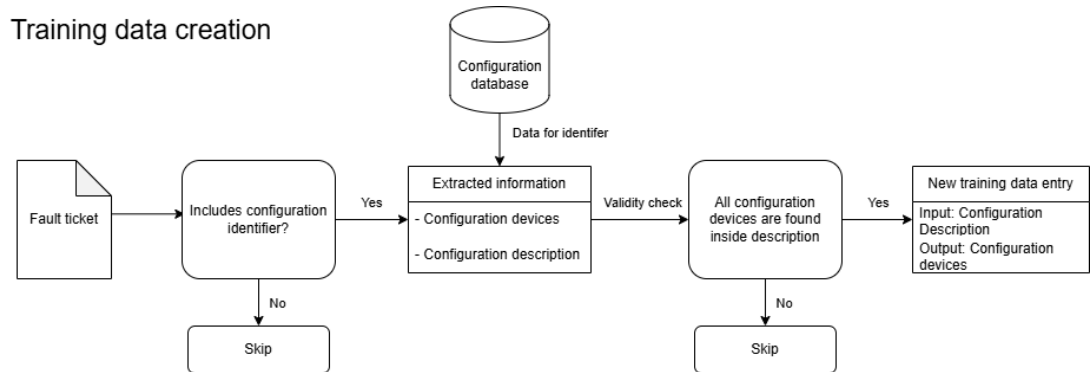


Figure 3. Visualization on the intended data creation pipeline.

4.2. LLM Fine-Tuning

The training was done on a distributed computing cluster. This is important since it caused some variations in the training speed between runs. The optimizer chosen for this fine-tuning was AdamW, which in comparison to the Adam optimizer separates weight decay from gradient update. This distinction helps AdamW avoid overfitting by limiting large weights, which is a significant problem with a small unbalanced dataset

[20]. Epoch loss was investigated, and an epoch of 10 was chosen based on the epoch loss curve shown in Figure 4.

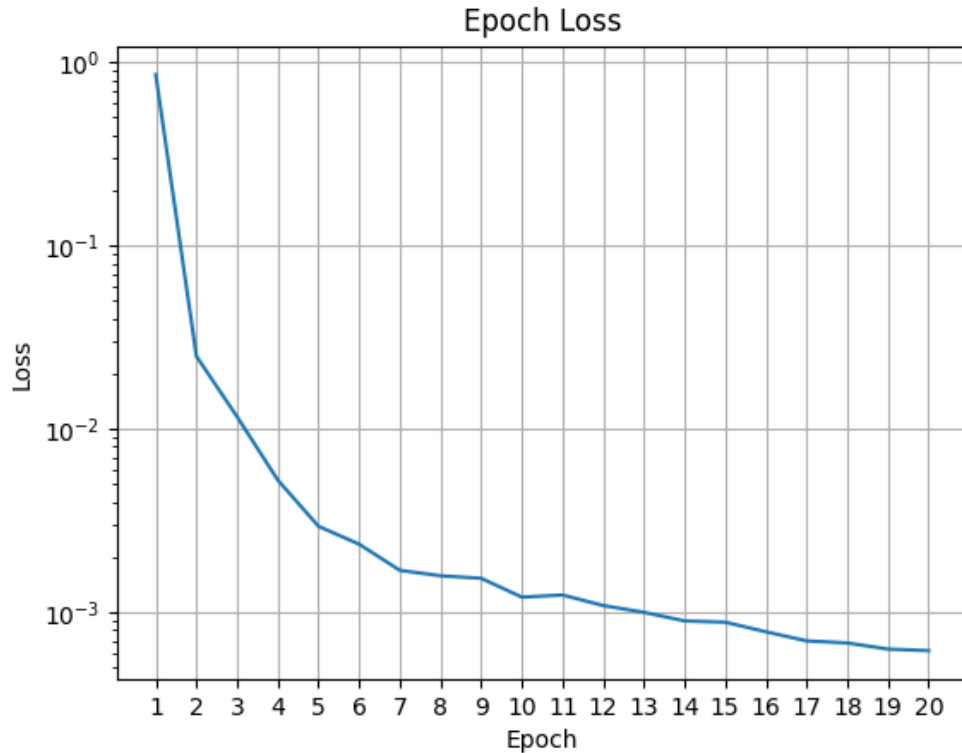


Figure 4. Training loss graph.

4.3. Context Reduction

In long context scenarios LLM's face several challenges including computational cost, performance reduction, and position bias [21]. It is important to note that this study researched far larger context amounts than the fault tickets and therefore the benefits might not be as drastic. The principle behind fault ticket context reduction is to find the references to the device being extracted and exclude information that is too distant from those references. For example, a large configuration might only mention the device once, and the rest of the ticket mostly talks about the interfaces and internal factors of that configuration. Thus, the LLM can focus only on the relevant information, which in theory mitigates the effects of position bias and improves performance. In this case, context reduction was achieved with a simple 50-character cutoff, and if there were multiple references, all information between them was retained.

4.4. Evaluation Methodology

Performance is judged based on the classification accuracy with which the models identify both the extraction device and the associated count if either is misidentified,

the result is counted as a failure. K-fold cross-validation was used to evaluate performance with a K-value of 5. In 5-fold cross-validation, the dataset is divided into five equal folds, with one fold being the validation set, and remaining were used for training. The dataset contains 850 records, meaning each fold has 170 records. This process is repeated five times, meaning that each fold is being evaluated and the results are averaged to obtain a more reliable performance estimate. This averaged estimate is then shown as an accuracy rate along with the standard deviation between the folds. Fine-tuning and evaluation was performed on the full and context reduced dataset. Equation 1 shows the formula for calculating the mean classification accuracy for K-folds and Equation 2 shows the formula for calculating classification standard deviation for K-folds, as follows

$$\text{Average accuracy} = \mu = \frac{1}{K} \sum_{i=1}^K \frac{\text{Number of correct extractions in fold } i}{\text{Total number of records in fold } i}, \quad (1)$$

and

$$\text{Standard deviation} = \sigma = \sqrt{\frac{1}{K} \sum_{i=1}^K (\text{Accuracy in fold } i - \mu)^2}. \quad (2)$$

Both formulae require equal fold sizes; otherwise the averages have to be weighted.

4.5. Evaluated Models

For evaluation, Regex-based approach was used as a baseline. It takes into account the occurrences of each of the words. In addition, it accounts for multiplication formats like "2x", "*2", "2" and simple written multipliers like "pieces of" and simple variations of it. Secondly, the corresponding pre-trained T5-variants was used as a second baseline to determine whether the fine-tuning made a difference. In addition, various model sizes were tested to establish whether model size has an impact on the ability to complete the task after and before fine-tuning.

4.6. Analysis

The mean classification accuracy and standard deviation was collected for all models and datasets as shown in Table 4. T5 base model was the best performing model in both full context and context reduced datasets. Variations between model performance was decreased after fine-tuning, which shows improvements in consistency between the different models. On average, context reduction performed slightly better with almost all trained models. Context reduction also improved performance compared to the full context version even when the dataset used was mostly made of relatively short descriptions. The context reduction was not perfect, which is apparent in the Regex scores which fell after context reduction. Regex was surprisingly effective in extracting configurations from fault tickets, but the fine-tuned large language model still outperformed it by over 10%. The fine-tuned LLM proved to be a more effective and general solution than a rule-based Regex approach. This is significant since the

dataset represents just a fraction of all the failure tickets, which likely will introduce even more unseen formats. Fine-tuning also improved the models performance significantly compared to the corresponding base model. A part of this performance increase came from the ability to answer in a structured format which the model learned extremely well. In testing, the model returned the answer in structured format with almost complete precision. Context reduction improved the models performance and effectiveness, but the method is still significantly slower than Regex, which benefits from an inherent algorithmic efficiency.

Table 4. Cross-validated classification accuracy results for device extraction on full and context reduced dataset. In the table μ represents mean and σ standard deviation

Model	Full [μ (σ)]		Context [μ (σ)]	
	Pre-trained	Fine-tuned	Pre-trained	Fine-tuned
T5-Small	0.470 (0.009)	0.865 (0.099)	0.469 (0.040)	0.898 (0.089)
T5-Base	0.436 (0.020)	0.923 (0.034)	0.494 (0.042)	0.935 (0.044)
T5-Large	0.180 (0.037)	0.919 (0.025)	0.458 (0.058)	0.922 (0.040)
T5-Flan-small	0.100 (0.019)	0.870 (0.121)	0.369 (0.037)	0.879 (0.120)
T5-Flan-base	0.113 (0.034)	0.889 (0.087)	0.333 (0.026)	0.897 (0.081)
T5-Flan-large	0.330 (0.018)	0.916 (0.036)	0.311 (0.035)	0.875 (0.094)
Regex	0.790 (0.016)	-	0.787 (0.023)	-

5. APPLICATION

In the second part, an application was created that merges together the data sources presented in Chapter 3. The application consists of different features. For the LLM based device extraction research a graphical user interface, shown in Figure 5, was created. Inside the GUI the user is able to input a fault ticket ID. The application then fetches that fault ticket and extracts the devices from it. This extracted device configuration is then shown to the user, who can add and remove devices from it. When ready, the user is able to compare the configuration devices to available devices inside the device inventory. Devices colored red are not available at all, orange are devices that are available but not at the quantity required. Green devices are fully available inside the inventory. Along with this, a percentage score is calculated to show the compatibility rating of the configuration in terms of available devices. The second feature, shown in Figure 6, allows the user to search topologies and the corresponding test-cases. After finding one or multiple, the user is able to fetch the current availability of that topology. If there are conflicts, the application shows a link to the running test and reservation along with a time estimate of the availability. This is not as simple as it seems since increasingly many devices are abstracted to resource pools, which allow for more dynamic and efficient usage of these devices. One of the use cases for this topology conflict checking is to ensure that more critical quality test-cases can be easily prioritized through a simple lookup rather than deducing these queued resources by hand. In addition to this, a few smaller features like test-case and reservation viewing page were created. In the future, the application could support further analysis through a test result database. The whole data pipeline is shown in Figure 7.

5.1. Application Infrastructure

A React-based web application was created to integrate the system with existing workflows. React was chosen because it offers components, declarative programming features, and an expansive ecosystem [22]. The frontend development included Vite [23] for build tooling and Typescript [24] for type safety. The backend is Python-based, as some of the previously created interfaces require the language. The Flask Web framework was chosen for its simplicity and flexibility [25]. The infrastructure is hosted in three different Docker containers; the frontend, backend, and the LLM container. This design choice offers excellent separation between resources, which helps with scaling, maintenance, deployment, security, and resource allocation. For example, the LLM container does not unnecessarily slow down the backend. Docker was used for ease of deployment, portability, dependency management, and its networking features. The containers communicate through a Docker network using the built-in domain name system (DNS). Figure 8 shows the infrastructure and the flow of data within it. The application is hosted only on an internal network, but authentication, password salting, and encryption were still implemented.

Reservations
Devices
Topologies
Testcases
Compatibility
Settings

Compatibility checking

Dev1	+	1 / 9	-	Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus pretium tortor sed lectus lobortis, quis vestibulum est vehicula. Quisque malesuada consequat diam, at tempor odio pretium porttitor. Nullam euismod ultricies maximus. Vestibulum euismod, tortor at malesuada accumsan, lectus elit rutrum augue, consequat feugiat neque ante a elit. Donec imperdiet ipsum nec quam rutrum lacinia. Vestibulum efficitur augue tortor, eu varius urna fermentum id. Vivamus quis fermentum ligula. Nulla lorem nunc, fringilla finibus dapibus et, convallis quis augue. Donec semper cursus sapien, et sodales nulla ullamcorper a. Vivamus lobortis, orci quis condimentum sagittis, turpis
Dev2	+	3 / 7	-	
Dev3	+	3 / 1	-	
Dev4	+	1 / 0	-	
Dev5	+	1 / 2	-	
Dev6	+	2 / 4	-	

Score: 72.73%

Found links:

https://testline_info/preview/
https://fault_ticket/preview

Figure 5. Compatibility checking interface with redacted information.

Reservations
Devices
Topologies
Testcases
Compatibility
Settings

Topology

Configuration 1

Estimated start time:

Available now

Topology

Configuration 2

Estimated start time

14:50:47 3.22.2025 (30 mins)

Conflicts:

Conflict 1: Topology 1

[Test link](#)
[Device inventory link](#)

Topology

Configuration 3

Estimated start time:

Available now

Figure 6. Conflict checking interface with redacted information.

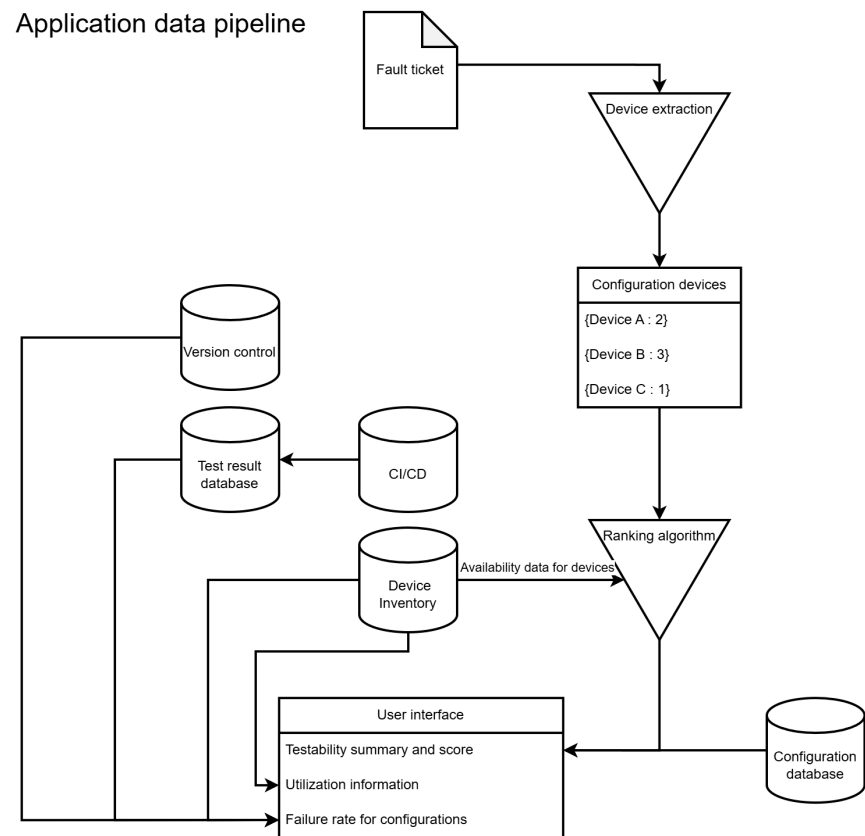


Figure 7. Visualization on the application data pipeline.

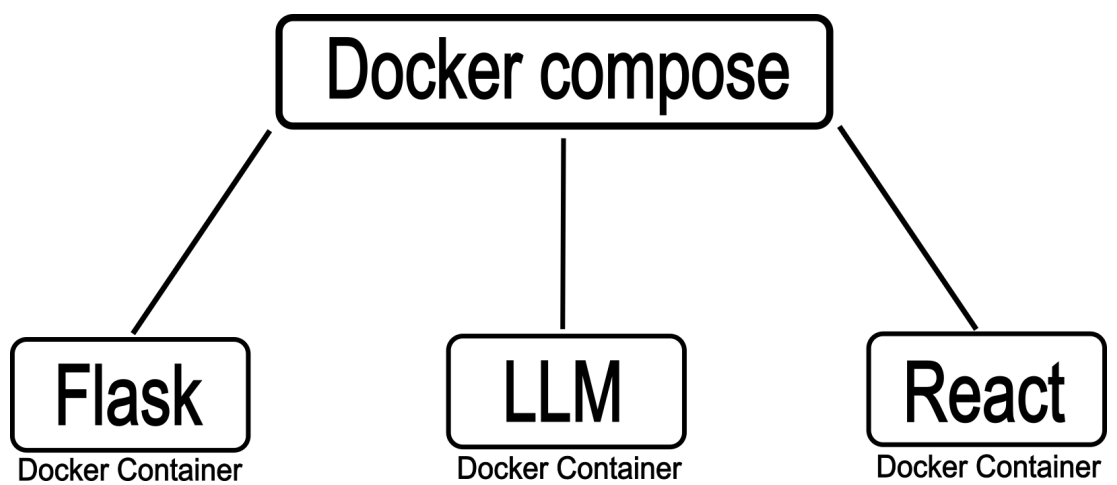


Figure 8. Basic view of the infrastructure.

6. DEVICE INVENTORY

The current device inventory allows for extensive programmatic usage, which provide critical aids for testing. While it has some faults and missing features that could cause issues – primarily missing API routes - it does bring some key benefits. After extensive usage and testing, key findings for larger adoption center around access groups. The device inventory allows for various kinds of access rights and groupings for devices. The device inventory maintains records of not only devices but also ports and cables and their connections – now referred to as units. Each unit has an owner, it can have shared users, shared user-groups, creator and creator-group. Furthermore, units can be grouped into resource groups, which themselves can have shared users, shared user-groups, owners, and creator-groups. Therefore the access rights and groupings can be done in various ways. As more teams start to use the device inventory, a common standard should be developed on how these access rights and ownership records are created. When implemented correctly, devices are simple to group by teams, utilization can be easily mapped to users and teams, and access rights are straightforward to change.

Additional requirements come from loaning devices, the device inventory must always know the original owner and allow for dynamic sharing. An example of dynamic sharing are teams that do testing during nighttime. They should be able to reserve equipment from teams that don't utilize these devices at night, but during the day the inventory should stop reservations to prevent conflicts. Figure 9 shows the resource ownership structure with resource groups. There a team would have one resource group which contains all their units. Each team would also have an user group which all team members would be a part of. This resource group would be then shared for the user group. If there is need for loaning units, a new loaning resource group would be created and shared to another teams user group. This way the relations stay relatively simple, access rights are easy to adjust dynamically since just removing access rights to that loaning group is enough to remove access to all ports, devices and connections.

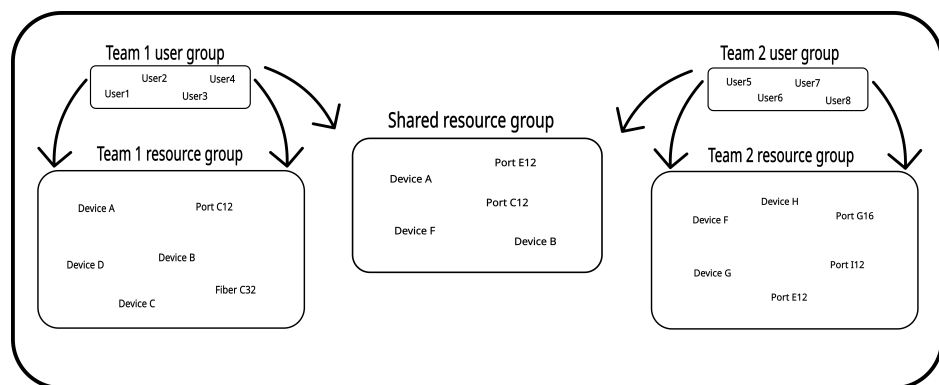


Figure 9. Visualization on how resource permissions could be organized with resource groups.

7. DISCUSSION

The fine-tuned T5 model proved to be an effective tool for extracting device configurations from natural language. It significantly outperformed the Regex counterpart, and the model's performance was greatly enhanced after fine-tuning. The fine-tuned model for extracting device configurations achieved performance comparable to the fine-tuned T5 model for location name extraction, with accuracies of approximately 93.5% and 93%, respectively [3].

One of the key limitations for fine-tuning was the relatively small amount of training data. Some other approaches like synthetic data generation and unsupervised learning could be utilized to circumvent this problem. Furthermore, dataset curation and hyperparameter tuning could be further examined. Model size within the T5 model family didn't affect performance but more powerful, models other than T5 could also be tested and utilized. Although, one of the benefits with T5-base is its small performance requirements. A larger model would then require either a more powerful machine or a separate machine with support for a graphics processing unit (GPU) depending on the models hardware requirements.

As the application is used, it can continuously collect more training data from user-verified configurations. This could lead to further improvements in the long term. In the future, the application would be able to support even more functionalities. One of these could be a user friendly way to visualize and edit permissions for the groups. The application could also support managing reservation priorities and cancellations. In the future, the fault ticket compatibility checking could introduce recommendations for borrowing devices from other teams. This could drastically help in sourcing parts for more complex testing configurations.

For the device inventory, some key limitations were identified and solved. The conflicts between ongoing and queued reservations could not be directly fetched from the API but an algorithm was implemented to alleviate this problem. After studying the device inventory, a recommendation was created for organizing devices and their permissions inside the device inventory. This could help avoid problems with future integrations and inventory management.

8. SUMMARY

This thesis investigated whether device configurations, depicted in natural language, could be effectively extracted with a LLM-based approach. Specifically, focusing on whether it can outperform a more traditional rule-based approach, and fine-tuning enhances the extraction performance. In addition, this thesis aimed to find out limitations and considerations that should be taken into account with the device inventory. With a fine-tuned LLM-based device extraction process, noticeable improvements were gained compared to Regex and the pretrained LLM counterpart. The extraction accuracy increased to 94% with the fine-tuned T5-base model compared to Regex with 79% and pre-trained T5-base models 50% accuracy. After fine-tuning the model was able to output in a structured format with excellent accuracy. The application built around this offered an interface for discovering testable configurations and detecting conflicts. After studying the device inventory, the key findings came from the permission and ownership system. A standard for should be created before larger adoption, as the inventory allows for multiple different approaches for organizing these devices.

9. REFERENCES

- [1] Kazi B.U. & Wainer G.A. (2019) Next generation wireless cellular networks: ultra-dense multi-tier and multi-cell cooperation perspective. *Wireless Networks* 25, pp. 2041–2064.
- [2] Zhang W., Wang Q., Kong X., Xiong J., Ni S., Cao D., Niu B., Chen M., Li Y., Zhang R., Wang Y., Zhang L., Li X., Xiong Z., Shi Q., Huang Z., Fu Z. & Zheng M. (2024) Fine-tuning large language models for chemical text mining. *Chem. Sci.* 15, pp. 10600–10611. URL: <http://dx.doi.org/10.1039/D4SC00924J>.
- [3] Dahlan A.F. & Yuangyai C. (2024) T5-based named entity recognition for social media: A case study for location extraction. In: 2024 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology (IAICT), pp. 354–359.
- [4] Jiang J. (2012) *Information Extraction from Text*, Springer US, Boston, MA. pp. 11–41. URL: https://doi.org/10.1007/978-1-4614-3223-4_2.
- [5] Michael L.G., Donohue J., Davis J.C., Lee D. & Servant F. (2019) Regexes are hard: Decision-making, difficulties, and risks in programming regular expressions. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 415–426.
- [6] Parthasarathy V.B., Zafar A., Khan A. & Shahid A. (2024), The ultimate guide to fine-tuning llms from basics to breakthroughs: An exhaustive review of technologies, research, best practices, applied research challenges and opportunities. URL: <https://arxiv.org/abs/2408.13296>.
- [7] Jung S.J., Kim H. & Jang K.S. (2024) Llm based biological named entity recognition from scientific literature. In: 2024 IEEE International Conference on Big Data and Smart Computing (BigComp), pp. 433–435.
- [8] Liu Y., Li D., Wang K., Xiong Z., Shi F., Wang J., Li B. & Hang B. (2024) Are llms good at structured outputs? a benchmark for evaluating structured output capabilities in llms. *Information Processing & Management* 61, p. 103809. URL: <https://www.sciencedirect.com/science/article/pii/S0306457324001687>.
- [9] Eraytsev E., Kor. Available at: <https://github.com/eyurtsev/kor>. Accessed: Apr. 7, 2025.
- [10] lrgs, jsonformer. Available at: <https://github.com/lrgs/jsonformer>. Accessed: Apr. 7, 2025.
- [11] Team L., Structured output. Available at: https://python.langchain.com/v0.1/docs/modules/model_io/chat/structured_output. Accessed: Apr. 7, 2025.

- [12] Raffel C., Shazeer N., Roberts A., Lee K., Narang S., Matena M., Zhou Y., Li W. & Liu P.J. (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21, pp. 1–67. URL: <http://jmlr.org/papers/v21/20-074.html>.
- [13] Devlin J., Chang M.W., Lee K. & Toutanova K.N. (2018) Bert: Pre-training of deep bidirectional transformers for language understanding. In: *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. URL: <https://arxiv.org/abs/1810.04805>.
- [14] Chung H.W., Hou L., Longpre S., Zoph B., Tay Y., Fedus W., Li Y., Wang X., Dehghani M., Brahma S., Webson A., Gu S.S., Dai Z., Suzgun M., Chen X., Chowdhery A., Castro-Ros A., Pellat M., Robinson K., Valter D., Narang S., Mishra G., Yu A., Zhao V., Huang Y., Dai A., Yu H., Petrov S., Chi E.H., Dean J., Devlin J., Roberts A., Zhou D., Le Q.V. & Wei J. (2024) Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, pp. 1–53. URL: <http://jmlr.org/papers/v25/23-0870.html>.
- [15] Research G. (2020), Text-to-text transfer transformer. URL: <https://github.com/google-research/text-to-text-transfer-transformer>, pre-trained model and codebase for text-to-text transfer learning.
- [16] Google (2022), Flan-t5 large. URL: <https://huggingface.co/google/flan-t5-large>, a fine-tuned version of T5-Large, demonstrating instruction tuning for text-to-text tasks.
- [17] Zolkifli N.N., Ngah A. & Deraman A. (2018) Version control system: A review. *Procedia Computer Science* 135, pp. 408–415. URL: <https://www.sciencedirect.com/science/article/pii/S1877050918314819>, the 3rd International Conference on Computer Science and Computational Intelligence (ICCSCI 2018) : Empowering Smart Technology in Digital Era for a Better Life.
- [18] Saranathan G., Alam M.P., Lim J., Bhattacharya S., Wong S.Y., Foltin M. & Xu C. (2024) DELE: Data efficient LLM evaluation. In: *ICLR 2024 Workshop on Navigating and Addressing Data Problems for Foundation Models*. URL: <https://openreview.net/forum?id=I8bsxPWLNF>.
- [19] Dina A.S., Siddique A.B. & Manivannan D. (2022) Effect of balancing data using synthetic data on the performance of machine learning classifiers for intrusion detection in computer networks. *IEEE Access* 10, pp. 96731–96747.
- [20] Nabila P. & Setiawan E.B. (2024) Adam and adamw optimization algorithm application on bert model for hate speech detection on twitter. In: *2024 International Conference on Data Science and Its Applications (ICoDSA)*, pp. 346–351.
- [21] Jiang H., Wu Q., Luo X., Li D., Lin C.Y., Yang Y. & Qiu L. (2024), Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. URL: <https://arxiv.org/abs/2310.06839>.

- [22] Meta Platforms I., React. Available at: <https://react.dev/>. Accessed: Apr. 9, 2025.
- [23] Vite, Vite. Available at: <https://vite.dev>. Accessed: Apr. 21, 2025.
- [24] Microsoft, Typescript - javascript that scales. Available at: <https://www.typescriptlang.org/>. Accessed: Apr. 21, 2025.
- [25] Pallets, Flask. Available at: <https://flask.pallets.com/en/stable>. Accessed: Apr. 7, 2025.